LAPORAN TUGAS KECIL II

IF2211 STRATEGI ALGORITMA

VOXELIZATION OBJEK 3D MENGGUNAKAN OCTREE

Disusun Oleh :

Natanael I. Manurung 13524021

Marcel Luther Sitorus 13524063

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

MARET 2026

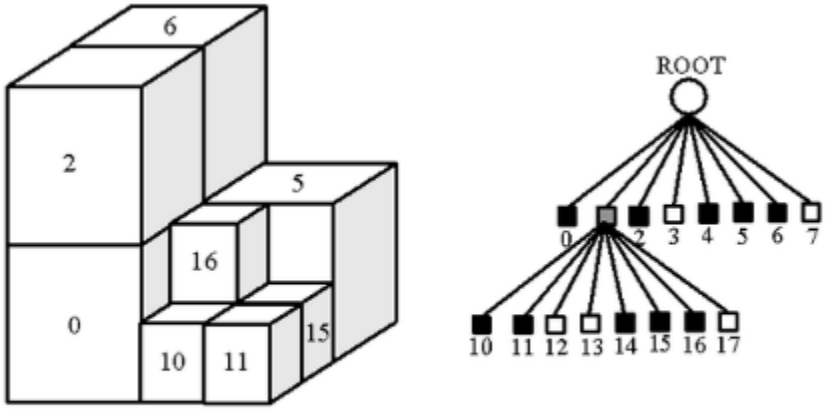
DAFTAR ISI

DAFTAR ISI **2**
BAB I **3**
BAB II **4**
1. Gambaran Umum Algoritma 4
2. Parsing File .OBJ (Tahap 1) 4
3. Menghitung Root Bounding Cube (Tahap 2) 6
4. Algoritma Divide and Conquer (Tahap 3) 7
4.1. Struktur Rekursi 7
4.2. Subdivisi: Membagi Kotak Menjadi 8 Oktant 8
4.3. Pruning: Mengapa Divide and Conquer Efisien 9
5. Uji Interseksi: Separating Axis Theorem (SAT) (Tahap 4) 10
5.1. Tiga Kelompok Sumbu SAT 10
5.2. Kenapa 13 Sumbu Sudah Cukup 11
6. Pembangkitan Geometri Voxel (Tahap 5) 12
7. Bonus: Concurrency dengan Goroutine 13
7.1. Strategi Hybrid Parallel/Sequential 13
BAB IV **16**
LAMPIRAN **17**

BAB I

DESKRIPSI TUGAS

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data Octree, yaitu sebuah tree yang setiap percabangan terdiri dari 8 anak. Melalui Octree, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.



Gambar 2 Visualisasi *Octree* terhadap 3D Model
Sumber:

https://www.researchgate.net/figure/a-Octants-b-Octree-Structure-c-Linear-Octree-Each-of-the-cubes-has-a-locational-code_fig1_221115507

Tugas Anda adalah mengimplementasikan sebuah program yang dapat mengkonversi model .obj yang umum menjadi model .obj yang terdiri dari voxel dengan memanfaatkan properti Octree. Program melakukan konversi model dengan menggunakan algoritma divide and conquer.

BAB II

LANDASAN TEORI

1. Gambaran Umum Algoritma

Secara keseluruhan, algoritma terdiri dari lima tahap utama yang dieksekusi secara berurutan. Setiap tahap memiliki peran yang jelas dan saling bergantung satu sama lain.

Lima Tahap Utama Algoritma

1. Parse file .obj = dapatkan daftar verteks dan segitiga
2. Hitung root bounding cube = kotak terkecil yang membungkus seluruh model
3. Divide and Conquer (Octree) = rekursi membagi ruang dan menyaring segitiga
4. Uji interseksi SAT = tentukan apakah segitiga menyentuh suatu kotak
5. Bangkitkan geometri voxel = tulis kubus-kubus ke file output .obj

2. Parsing File .OBJ (Tahap 1)

File .obj adalah format teks sederhana. Program membaca baris demi baris dan memproses hanya dua jenis baris:

- Baris v x y z mendefinisikan satu verteks dengan koordinat floating point.
- Baris f i j k ... mendefinisikan satu face berupa indeks ke daftar verteks (1-based). Face dengan lebih dari 3 sudut ditriangulasi menggunakan fan triangulation.

Validasi yang dilakukan:

- Baris v harus memiliki tepat 3 nilai float yang parseable.
- Baris f harus memiliki setidaknya 3 indeks; setiap indeks harus dalam rentang [1, jumlah verteks].
- File tidak boleh kosong harus ada minimal 1 verteks dan 1 face.
- Indeks negatif (referensi relatif) ditangani dengan konversi ke indeks absolut.

Pseudocode parser:

```
function PARSE_OBJ(path):
    vertices <- []
    faces    <- []
```

```
    for line in file:
        tokens <- split(line)
        if tokens[0] == 'v':
            x, y, z <- parse_float(tokens[1..3])
            vertices.append(Vec3(x, y, z))

        else if tokens[0] == 'f':
            indices <- []
            for token in tokens[1:]:
                  idx <- parse_int(split(token, '/')[0])    //
ambil hanya vertex index
                   if idx < 0: idx <- len(vertices) + idx + 1
// negatif =absolut
                  indices.append(idx - 1)                    //
konversi ke 0-based

               // Fan triangulation untuk polygon dengan > 3
sudut
            for i in 1 .. len(indices)-2:
                 faces.append(Triangle(indices[0], indices[i],
indices[i+1]))

    return Model(vertices, faces)
```

setelah parsing, face dikonversi dari indeks ke koordinat aktual. Ini menghasilkan array rawTriangle yang menyimpan tiga posisi 3D secara langsung, sehingga setiap uji interseksi nantinya tidak perlu melakukan dereferencing indeks.

```
// Dari octree.go  resolusi face ke koordinat
tris := make([]rawTriangle, len(model.Faces))
for i, f := range model.Faces {
    tris[i] = rawTriangle{
        V0: model.Vertices[f.A],
        V1: model.Vertices[f.B],
```

```
        V2: model.Vertices[f.C],
    }
}
```

3. Menghitung Root Bounding Cube (Tahap 2)

Sebelum rekursi dimulai, program menghitung satu kotak pembungkus akar (root bounding box) yang mencakup seluruh model. Kotak ini harus berupa kubus sempurna sisi yang sama pada ketiga sumbu X, Y, Z agar setiap voxel di leaf level juga merupakan kubus yang seragam (uniform).

Langkah-langkah:

- Iterasi semua verteks dan cari nilai min/max pada setiap sumbu.
- Hitung panjang sisi pada tiap sumbu: dx, dy, dz.
- Ambil panjang sisi terbesar: maxD = max(dx, dy, dz).
- Hitung centroid model: (cx, cy, cz).
- Bangun kubus dengan setengah sisi = maxD/2 + epsilon, dicentrasi di centroid.
- Epsilon kecil (1e-6) ditambahkan agar segitiga yang tepat di batas tidak terlewat.

```
// Dari geometry.go  ComputeBounds
function COMPUTE_ROOT_CUBE(vertices):
    minX, minY, minZ <- +∞
    maxX, maxY, maxZ <- -∞

    for v in vertices:
        minX <- min(minX, v.X);  maxX <- max(maxX, v.X)
        minY <- min(minY, v.Y);  maxY <- max(maxY, v.Y)
        minZ <- min(minZ, v.Z);  maxZ <- max(maxZ, v.Z)

    dx <- maxX - minX
    dy <- maxY - minY
    dz <- maxZ - minZ
    maxD <- max(dx, dy, dz)

    cx <- (minX + maxX) / 2
```

```
cy <- (minY + maxY) / 2
cz <- (minZ + maxZ) / 2
half <- maxD / 2 + 1e-6        // epsilon

return AABB(
    Min: Vec3(cx - half, cy - half, cz - half),
    Max: Vec3(cx + half, cy + half, cz + half)
)
```

Mengapa harus kubus? Jika kotak tidak seragam (misalnya lebih panjang di sumbu X), maka setiap subdivisi akan menghasilkan kotak-kotak dengan dimensi berbeda di setiap oktant, dan voxel di leaf level akan memiliki ukuran berbeda-beda. Padahal salah satu syarat di spek adalah output voxel harus berukuran uniform.

4. Algoritma Divide and Conquer (Tahap 3)

Ini adalah inti dari seluruh program. Algoritma menggunakan paradigma Divide and Conquer untuk membangun Octree secara rekursif. Setiap node pada Octree merepresentasikan sebuah kotak kubik di dalam ruang 3D.

4.1. Struktur Rekursi

Setiap pemanggilan rekursif menerima:

- kotak kubik yang sedang diproses
- daftar segitiga yang mungkin berpotongan dengan kotak ini (sudah difilter dari level atas)
- kedalaman node saat ini
- kedalaman maksimum yang diperbolehkan

Dan menjalankan tiga kemungkinan aksi:

```
function BUILD_OCTREE(box, triangles, depth, maxDepth):
  // Langkah 1  Filter: hanya pertahankan segitiga yang menyentuh box ini
  overlapping <- FILTER_TRIANGLES(triangles, box)
```

```
// Langkah 2  Base case A: tidak ada segitiga =PRUNE (hangus)
if len(overlapping) == 0:
    record_pruned(depth)
    return                    // tidak ada voxel di sini
// Langkah 3  Base case B: sudah di kedalaman maksimum =VOXEL
if depth == maxDepth:
    record_leaf(box)          // kotak ini menjadi satu voxel
    return
// Langkah 4  Divide: bagi box menjadi 8 oktant
octants[0..7] <- SUBDIVIDE(box)
// Langkah 5  Conquer: rekursi ke setiap oktant
for octant in octants:
    BUILD_OCTREE(octant, overlapping, depth + 1, maxDepth)
```

4.2. Subdivisi: Membagi Kotak Menjadi 8 Oktant

Proses divide dilakukan dengan memotong kotak tepat di titik tengahnya pada ketiga sumbu secara bersamaan. Titik potong adalah centroid dari kotak: c = (Min + Max) / 2.
Hasilnya adalah 8 kotak baru yang ukurannya tepat setengah dari kotak induk di setiap dimensi. Kedelapan oktant ini mencakup seluruh volume kotak induk tanpa tumpang tindih.

```
// Dari geometry.go  subdivide
function SUBDIVIDE(box):
    c <- box.Center()       // c = (box.Min + box.Max) / 2
    return [
         AABB(Min=(box.Min.X, box.Min.Y, box.Min.Z), Max=(c.X,
c.Y,        c.Z      )),  // 0: -X -Y -Z
              AABB(Min=(c.X,            box.Min.Y,  box.Min.Z),
Max=(box.Max.X, c.Y,        c.Z      )),  // 1: +X -Y -Z
         AABB(Min=(box.Min.X, box.Min.Y, c.Z      ), Max=(c.X,
c.Y,        box.Max.Z)),  // 2: -X -Y +Z
             AABB(Min=(c.X,           box.Min.Y,  c.Z         ),
Max=(box.Max.X, c.Y,        box.Max.Z)),  // 3: +X -Y +Z
```

```
        AABB(Min=(box.Min.X, c.Y,       box.Min.Z), Max=(c.X,
box.Max.Y, c.Z      )),  // 4: -X +Y -Z
            AABB(Min=(c.X,           c.Y,           box.Min.Z),
Max=(box.Max.X, box.Max.Y, c.Z      )),  // 5: +X +Y -Z
        AABB(Min=(box.Min.X, c.Y,       c.Z      ), Max=(c.X,
box.Max.Y, box.Max.Z)),  // 6: -X +Y +Z
           AABB(Min=(c.X,           c.Y,          c.Z        ),
Max=(box.Max.X, box.Max.Y, box.Max.Z)),  // 7: +X +Y +Z
    ]
```

4.3. Pruning: Mengapa Divide and Conquer Efisien

Kunci efisiensi algoritma ini adalah pruning. Setiap kali sebuah oktant tidak memiliki satupun segitiga yang berpotongan dengannya, seluruh subtree di bawahnya langsung diabaikan tanpa rekursi lebih lanjut.

Pertimbangkan model seperti bola: bagian interior bola tidak memiliki segitiga permukaan, begitu pula dengan sebagian besar ruang eksterior. Pruning memastikan bahwa hanya okltan yang mengandung atau berbatasan dengan permukaan model yang direkursi lebih lanjut.

Analisis Kompleksitas dengan Pruning

Tanpa pruning: setiap level menghasilkan 8x lebih banyak node =O(8^maxDepth) total node.

Dengan pruning: hanya node yang menyentuh permukaan yang diteruskan.

Untuk model padat (solid surface), jumlah voxel permukaan skala ~O(4^maxDepth).

Ini setara dengan O(N^(2/3)) di mana N adalah total voxel, jauh lebih efisien.

Statistik node yang dipruning dicetak di output CLI untuk transparansi.

5.4 Filter Segitiga per Node

Pada setiap level rekursi, sebelum membagi lebih lanjut, program memfilter daftar segitiga dari node induk untuk hanya meneruskan segitiga yang benar-benar berpotongan dengan kotak saat ini. Disinilah bagian conquernya, setiap submasalah bekerja hanya dengan data yang relevan.

```
// Dari octree.go  filterTriangles
```

```
function FILTER_TRIANGLES(triangles, box):
    result <- []
    for tri in triangles:
        if SAT_INTERSECTS(tri.V0, tri.V1, tri.V2, box):
            result.append(tri)
    return result
```

5. Uji Interseksi: Separating Axis Theorem (SAT) (Tahap 4)

Untuk memutuskan apakah sebuah segitiga berpotongan dengan sebuah kotak, program menggunakan Separating Axis Theorem (SAT), sebuah metode geometri yang bekerja berdasarkan prinsip: dua konveks tidak berpotongan jika dan hanya jika terdapat setidaknya satu sumbu pemisah yang memisahkan proyeksi keduanya.

5.1. Tiga Kelompok Sumbu SAT

SAT untuk segitiga vs AABB menguji 13 sumbu:

- 3 sumbu normal face AABB: X=(1,0,0), Y=(0,1,0), Z=(0,0,1).
- 1 sumbu normal face segitiga: cross product dari dua edge segitiga.
- 9 sumbu cross product: setiap 3 edge segitiga × setiap 3 sumbu AABB.

```
function SAT_INTERSECTS(v0, v1, v2, box):
    // Translasi: pusatkan kotak di origin
    c  <- box.Center()
    h  <- box.HalfSize()
 a0 <- v0 - c
    a1 <- v1 - c
    a2 <- v2 - c

    // Kelompok 1: 3 sumbu normal AABB
     if min(a0.X, a1.X, a2.X) > h.X or max(...) < -h.X: return
false  // X
     if min(a0.Y, a1.Y, a2.Y) > h.Y or max(...) < -h.Y: return
false  // Y
```

```
    if min(a0.Z, a1.Z, a2.Z) > h.Z or max(...) < -h.Z: return
false  // Z

    // Kelompok 2: 1 sumbu normal segitiga
    e0     <- a1 - a0
    e1     <- a2 - a1
    normal <- cross(e0, e1)
        r            <-  h.X*|normal.X|  +  h.Y*|normal.Y|  +
h.Z*|normal.Z|
    if |dot(normal, a0)| > r: return false

    // Kelompok 3: 9 sumbu cross product
    edges <- [e0, e1, a0 - a2]
    axes  <- [(1,0,0), (0,1,0), (0,0,1)]
    for e in edges:
        for ax in axes:
            sep  <- cross(e, ax)
                  p0, p1, p2 <- dot(sep, a0), dot(sep, a1),
dot(sep, a2)
            pMin <- min(p0, p1, p2)
            pMax <- max(p0, p1, p2)
            r    <- h.X*|sep.X| + h.Y*|sep.Y| + h.Z*|sep.Z|
            if pMin > r or pMax < -r: return false

    return true    // tidak ada sumbu pemisah =BERPOTONGAN
```

5.2. Kenapa 13 Sumbu Sudah Cukup

Untuk dua objek konveks, teorema menjamin bahwa jika terdapat sumbu pemisah, salah satunya pasti merupakan normal dari salah satu face, atau cross product dari pasangan edge keduanya. AABB memiliki 3 face normal dan 3 sumbu edge. Segitiga memiliki 1 face normal dan 3 edge. Sehingga jumlah total kandidat sumbu yang perlu diperiksa adalah: 3 (AABB normals) + 1 (tri normal) + 3×3 (cross products) = 13 sumbu.

Jika tidak ada satupun dari 13 sumbu ini yang memisahkan kedua objek, maka keduanya pasti berpotongan. Pengecekan berhenti segera begitu ada satu sumbu pemisah ditemukan (short-circuit).

6. Pembangkitan Geometri Voxel (Tahap 5)

Setelah rekursi selesai, program memiliki daftar leaf AABB aktif setiap AABB merepresentasikan satu voxel. Untuk setiap voxel, program membangkitkan geometri kubus yang terdiri dari 8 verteks dan 6 quad face.

Tata letak verteks pada setiap kubus voxel:

Verteks 0: (Min.X, Min.Y, Min.Z)	Verteks 4: (Min.X, Max.Y, Min.Z)
Verteks 1: (Max.X, Min.Y, Min.Z)	Verteks 5: (Max.X, Max.Y, Min.Z)
Verteks 2: (Max.X, Min.Y, Max.Z)	Verteks 6: (Max.X, Max.Y, Max.Z)
Verteks 3: (Min.X, Min.Y, Max.Z)	Verteks 7: (Min.X, Max.Y, Max.Z)

```
function EMIT_VOXEL_CUBE(box, result):
        base  <-  len(result.vertices)          //  offset  ke
verteks-verteks baru
    mn   <- box.Min
    mx   <- box.Max

    // 8 verteks
    result.vertices.append(
        (mn.X, mn.Y, mn.Z),  // 0
        (mx.X, mn.Y, mn.Z),  // 1
        (mx.X, mn.Y, mx.Z),  // 2
        (mn.X, mn.Y, mx.Z),  // 3
        (mn.X, mx.Y, mn.Z),  // 4
        (mx.X, mx.Y, mn.Z),  // 5
        (mx.X, mx.Y, mx.Z),  // 6
        (mn.X, mx.Y, mx.Z),  // 7
    )
```

```
b <- base
// 6 quad face (winding outward)
result.faces.append(
    (b+0, b+1, b+2, b+3),  // Bawah  (-Y)
    (b+7, b+6, b+5, b+4),  // Atas   (+Y)
    (b+0, b+4, b+5, b+1),  // Depan  (-Z)
    (b+2, b+6, b+7, b+3),  // Belakang (+Z)
    (b+0, b+3, b+7, b+4),  // Kiri   (-X)
    (b+1, b+5, b+6, b+2),  // Kanan  (+X)
)
```

Setiap voxel menghasilkan tepat 8 verteks dan 6 face. Untuk N voxel, output akan memiliki 8N verteks dan 6N face. Winding order dipilih agar normal face mengarah ke luar kubus, sehingga model dapat di-render dengan benar menggunakan backface culling.

7. Bonus: Concurrency dengan Goroutine

Untuk memenuhi spesifikasi bonus, program mengimplementasikan concurrency menggunakan primitif Go: goroutine, sync.WaitGroup, sync.Mutex, dan channel semaphore.

7.1. Strategi Hybrid Parallel/Sequential

Tidak semua level rekursi dieksekusi secara paralel. Program menggunakan strategi hybrid:
[object Object] Setiap 8 oktant diproses dalam goroutine terpisah secara simultan. Ini memberikan paralelisme nyata di mana setiap subtree cukup besar untuk mengamortisasi overhead goroutine.
[object Object] Rekursi kembali ke mode sequential. Di level bawah, jumlah node sangat banyak namun setiap node sangat ringan membuat goroutine akan membebani scheduler lebih dari yang dihemat.

Alasan pemilihan threshold depth 4:
Pada depth 4, sudah ada hingga 4.096 node aktif yang dapat berjalan paralel.
Pada depth 5+, jumlah node dapat mencapai jutaan goroutine overhead mengalahkan manfaat.

Semaphore membatasi goroutine hidup maksimum 512 secara bersamaan.

```
const PARALLEL_DEPTH_THRESHOLD = 4

function BUILD_OCTREE_CONCURRENT(box, tris, depth, maxDepth,
collector, sem):
    collector.addNode(depth)
    overlapping <- FILTER_TRIANGLES(tris, box)

    if len(overlapping) == 0:
        collector.addPruned(depth)
        return

    if depth == maxDepth:
        collector.addLeaf(box)     // thread-safe via mutex
        return

    octants <- SUBDIVIDE(box)

    if depth <= PARALLEL_DEPTH_THRESHOLD:
        // Paralel: setiap oktant dalam goroutine sendiri
        for octant in octants:
             sem.acquire()           // batasi goroutine aktif
maks 512
            go func(oct):
                defer sem.release()
                     BUILD_OCTREE_CONCURRENT(oct, overlapping,
depth+1, ...)
            end goroutine
        wait_all_goroutines()
    else:
          // Sequential: hindari overhead goroutine di level
dalam
        for octant in octants:
```

```
                    BUILD_OCTREE_CONCURRENT(octant,  overlapping,
depth+1, ...)
```

7.2. Thread Safety dengan concurrentCollector

Karena banyak goroutine menulis ke struktur data bersama, digunakan concurrentCollector yang melindungi setiap operasi write dengan sync.Mutex:

```
// Dari octree.go
type concurrentCollector struct {
    mu     sync.Mutex
    leaves []AABB   // daftar leaf voxel aktif
    nodes  []int    // jumlah node per depth
    pruned []int    // jumlah node dipruning per depth
}

func (c *concurrentCollector) addLeaf(box AABB) {
    c.mu.Lock()
    c.leaves = append(c.leaves, box)   // atomic write
    c.mu.Unlock()
}
```

Setelah seluruh rekursi selesai (semua WaitGroup done), fase pembangkitan geometri (Tahap 5) berjalan single-threaded karena tidak ada race condition di sana ini adalah tradeoff yang tepat karena append ke slice berurutan jauh lebih cepat daripada append paralel dengan locking.

BAB IV

SCREENSHOT

1. cow.obj

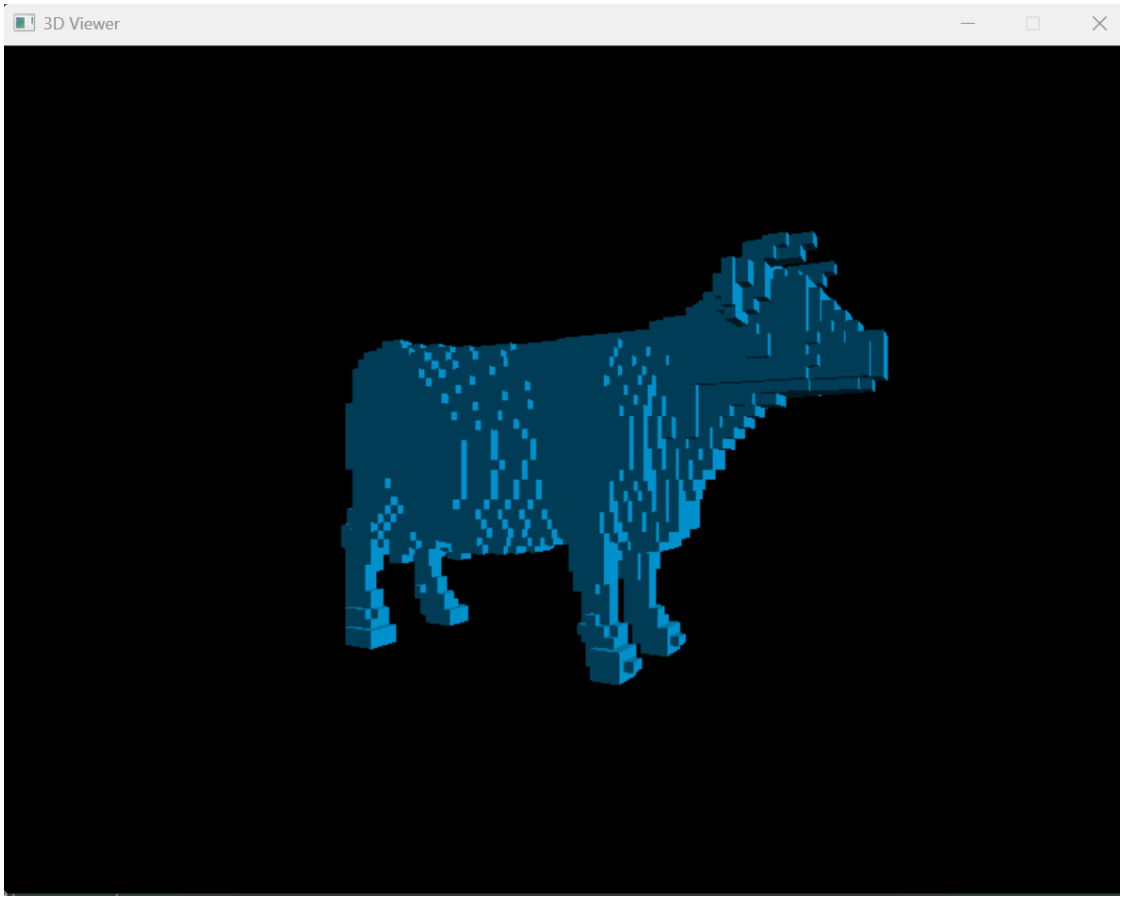
```
C:\Users\Lenovo\programming_project\CPP\Tucil2-Stima\Tucil2_13524063\src>voxelizer.exe "..\test\cow.obj" 7
Loading model: ..\test\cow.obj
Loaded 4583 vertices and 5804 faces.
Voxelizing with max depth 7...

=== Voxelization Report ===
Voxel count        : 5627
Vertex count       : 45016
Face count         : 33762
Octree depth       : 7

Octree nodes per depth:
  1 : 1
  2 : 8
  3 : 64
  4 : 160
  5 : 704
  6 : 2656
  7 : 10896

Pruned (skipped) nodes per depth:
  1 : 0
  2 : 0
  3 : 44
  4 : 72
  5 : 372
  6 : 1294
  7 : 5269

Elapsed time       : 22.0731ms
Output saved to    : ..\test\cow-voxelized.obj
View the result (y/n) ?
```


3D Viewer

2. line.obj

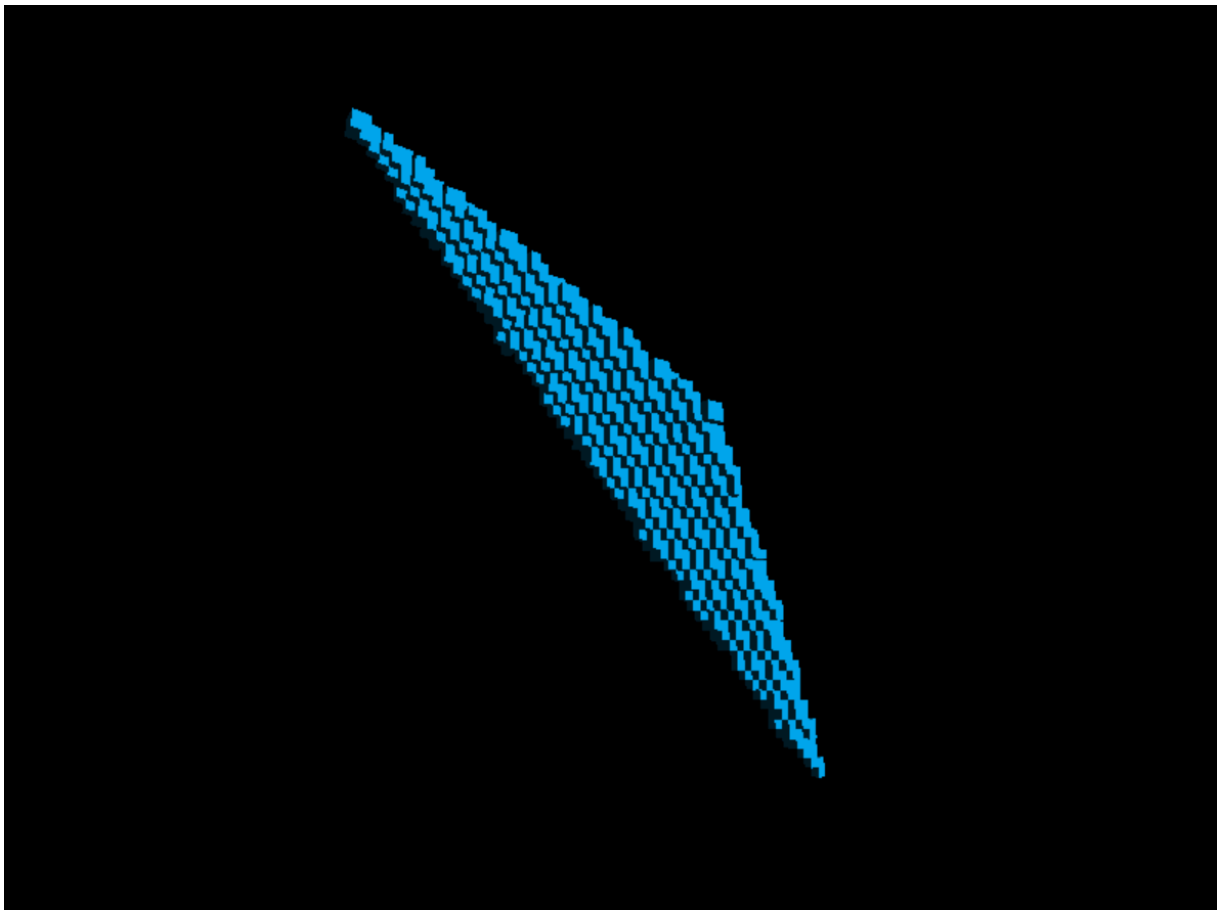
```
C:\Users\Lenovo\programming_project\CPP\Tucil2-Stima\Tucil2_13524063\src>voxelizer.exe "..\test\line.obj" 7
Loading model: ..\test\line.obj
Loaded 3 vertices and 1 faces.
Voxelizing with max depth 7...

=== Voxelization Report ===
Voxel count        : 1019
Vertex count       : 8152
Face count         : 6114
Octree depth       : 7

Octree nodes per depth:
  1 : 1
  2 : 8
  3 : 56
  4 : 120
  5 : 264
  6 : 744
  7 : 2304

Pruned (skipped) nodes per depth:
  1 : 0
  2 : 1
  3 : 41
  4 : 87
  5 : 171
  6 : 456
  7 : 1285

Elapsed time       : 2.646ms
Output saved to    : ..\test\line-voxelized.obj
View the result (y/n) ? y
```



3. **pumpkin.obj**

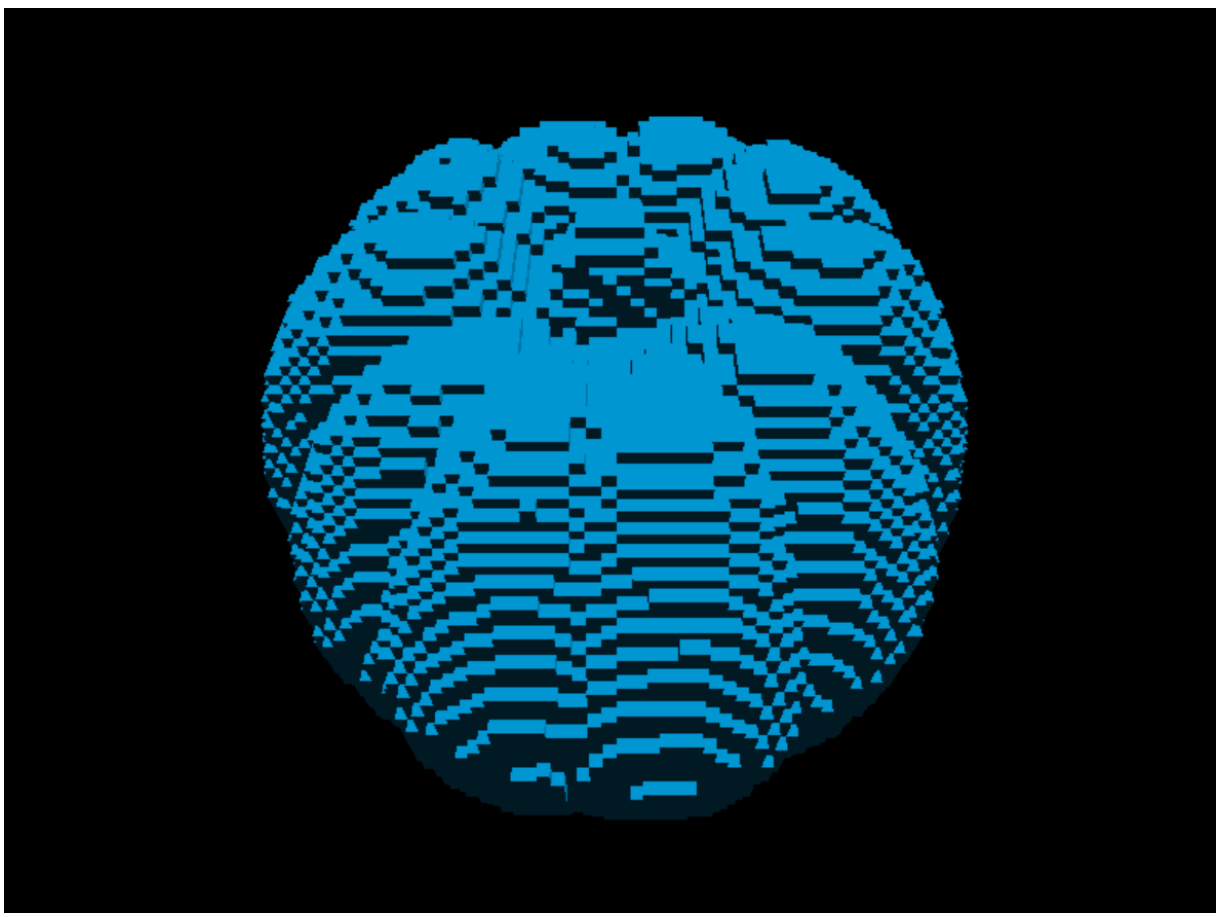
```
C:\Users\Lenovo\programming_project\CPP\Tucil2-Stima\Tucil2_13524063\src>voxelizer.exe "..\test\pumpkin.obj" 7
Loading model: ..\test\pumpkin.obj
Loaded 5002 vertices and 10000 faces.
Voxelizing with max depth 7...

=== Voxelization Report ===
Voxel count        : 18637
Vertex count       : 149096
Face count         : 111822
Octree depth       : 7

Octree nodes per depth:
  1 : 1
  2 : 8
  3 : 64
  4 : 448
  5 : 2216
  6 : 8984
  7 : 37008

Pruned (skipped) nodes per depth:
  1 : 0
  2 : 0
  3 : 8
  4 : 171
  5 : 1093
  6 : 4358
  7 : 18371

Elapsed time       : 60.9024ms
Output saved to    : ..\test\pumpkin-voxelized.obj
View the result (y/n) ? y
```



4. teapot.obj

LAMPIRAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Natanael I. Manurung

Marchel Luther Sitorus

Link github: https://github.com/LutherCeltors/Tucil2_13524021_13524063